

# RUST IN PEACE

May it be with you

Sergey Gordeychik (@scadastrangelove/@scadasl)  
CyberOK / SCADA StrangeLove



# Millions of packets. Wire speed.

## C / C++ / Rust

My shortlist for this project.  
Rust was new to me.  
Linux and Chromium already use it.

**And it is safe. What could possibly go wrong?**

[Linux: Rust](#) [Chromium CL 8183991](#)



# We don't know yet



# Ownership and borrowing

```
let a = vec![1, 2, 3];  
let b = a;  
println!("{}", a);
```

**Use after move:  
the compiler rejects it.**

## Ownership

A move transfers the value. The old owner cannot use it.

## Borrowing + lifetimes

References must stay valid. Shared and exclusive access have rules.

## Send / Sync

Constraints on values and references crossing thread boundaries.

**Memory guarantees without a GC. Sound unsafe abstractions remain a premise.**

[Ownership](#) [Send/Sync](#)

# The tools need something to test

**Cargo + crates**      Reusable packages become part of the application's trust boundaries.

**Tests**      Check the examples and assertions someone wrote.

**Miri**      Detects classes of undefined behaviour on executed Rust paths.

**cargo-fuzz**      Coverage-guided libFuzzer explores inputs through a harness you supply.

**Choose a real entry point and an observable failure condition.**

[Cargo](#) [Miri](#) [cargo-fuzz](#)

# Safe ≠ secure

## CVE-2024-24576

Rust standard library  
Windows .bat / .cmd arguments could  
escape Command quoting and execute  
commands.

## CVE-2023-26964

h2  
HEADERS / RST\_STREAM floods could  
grow a pending queue and exhaust  
memory.

**Command interpretation and resource accounting remain security boundaries.**

Published historical CVEs. These are separate from my findings.

[Rust std: CVE-2024-24576](#) [h2: CVE-2023-26964](#)

# Damn Vulnerable Rust Application

A small artifact-processing lab.

A threat model became my list of what must never happen.

**Early tool runs found none.  
That was a challenge.**

1. Cross-tenant access / IDOR
2. Command injection
3. Stale parser offsets
4. Panic-safety double drop
5. Invalid Sync implementation
6. Secrets in debug logs
7. Archive path traversal
8. SSRF to metadata
9. Unreachable legacy defect

**Nine planted defects, including unreachable legacy code. Plus one safe decoy.**

My first tools missed the bugs. Those early run logs were not preserved.

[DVRA application](#) [DVRA-3 gold labels](#)

# A threat model changed the experiment

Defending Code Reference Harness  
(DCRH).

9 / 9

I added a Rust profile and a working  
threat model.

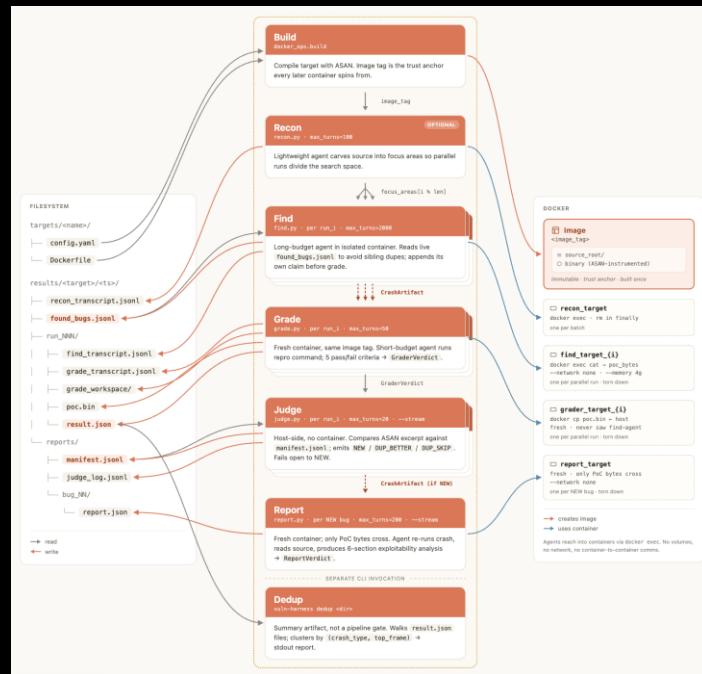
Later static recall  
hinted, neutral and blind variants  
with Rust profile + threat model

**Early hint-removal recall drop: recollection. Later 9/9: recorded static findings.**

The nine labels include unreachable legacy code. This is not nine crash PoCs.

[DCRH](#) [DVRA](#)

# The reference harness I started from



## Awesome AI Security Tools

A curated list of public-source, research, and commercial tools for AI security and AI-assisted cybersecurity — autotriage, agent security, AI/ML supply chain, pentest agents, AI SAST, LLM-driven fuzzing, threat intelligence, SOC/SIEM triage, reverse engineering, LLM red-teaming, and more.

license CC0-1.0 PRs welcome

Threat-modelled static review

C/C++ crash discovery with ASan

Parallel finders

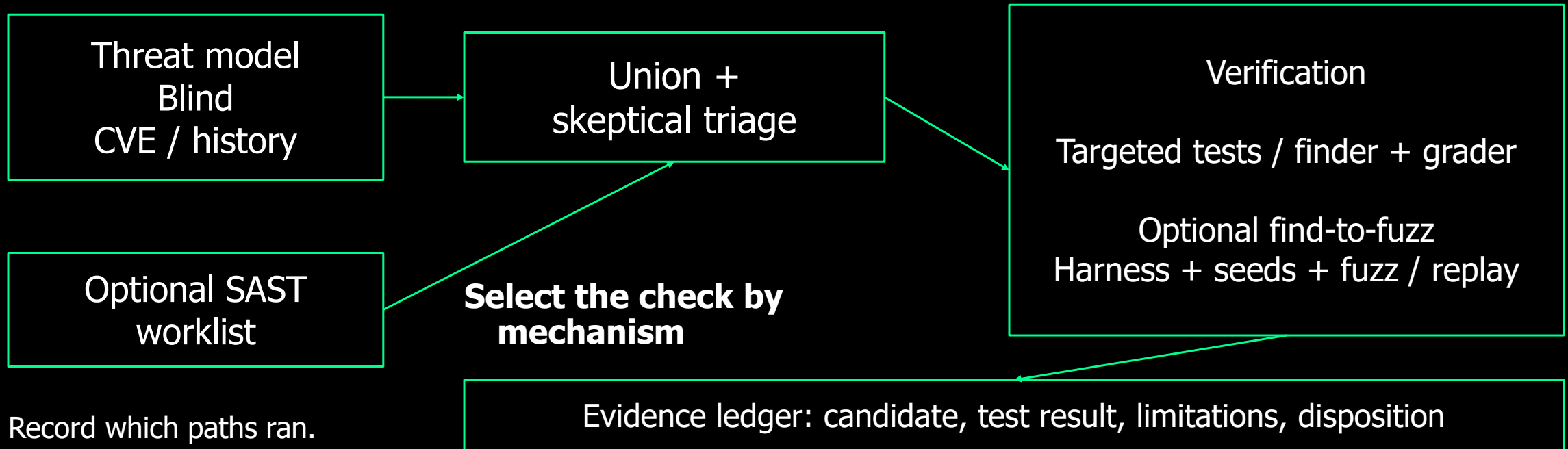
Fresh-container grading

Report and patch verification

**The execution and verification loop was already there.**

[Reference repository](#) [Original diagram](#)

# Raise: the rust-in-peace research system



**Independent search. Skeptical triage. Verification matched to the target.**

SAST and find-to-fuzz are optional branches. Static campaigns and the execution CLI are complementary paths.

[Research methods](#) [SAST](#) [Find → fuzz](#)

# Threat model: what must never happen?

## Does the limit apply before allocation?

Attacker inputs. Trust boundaries. API promises.  
Then trace every caller that relies on them.

**A useful working document also carries its author's blind spots.**

[Independent passes](#)

# Blind: leave, come back, start again

Same target. Fresh context.

No threat model, CVE seeds or neighbouring answers.

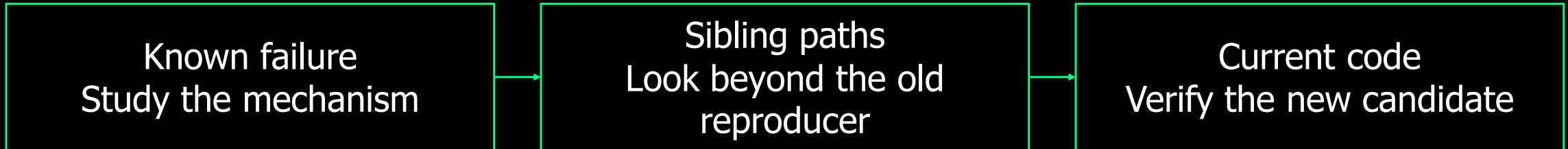
**“Step outside. Now come back in properly.”**

**Independent search first. Union and verification afterwards.**

quick-xml's extra blind signal later resolved as by-design / related to a known issue.

[quick-xml comparison](#)

# CVE / history: the same mistake elsewhere



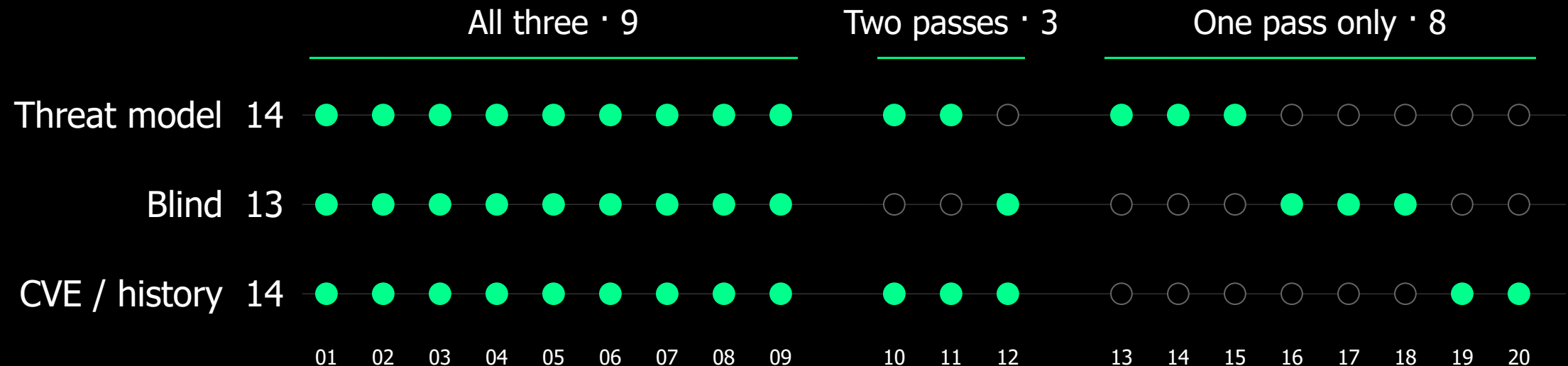
lopdf: limiting parser recursion did not bound later walks through the document graph.

**A fixed entry point does not prove every sibling path is fixed.**

[lopdf #530](#) [Fix #531](#)

# Different passes kept different findings

One column = one retained finding. Filled dot = surfaced in that pass.



**8 of 20 surfaced in only one pass.**

15 dynamic PoCs + 5 source-traced. Historical research cohort, not 20 shipping Chrome vulnerabilities.

[Research methods and counts](#)

# Two blind passes. No shared candidates.

Opus

Online-status panic #16339

Sonnet

Clipboard write #15693

Permission bypass #16333

| RustDesk   | Opus | Sonnet |
|------------|------|--------|
| Candidates | 4    | 6      |
| Retained   | 2    | 4      |
| Rejected   | 2    | 2      |

0 shared candidates. Union: 10 candidates, 6 retained.

Swarm here means independent models or lenses, then union + verification.  
3 Aug 2026 runs. Attribution: local logs. Retained evidence mixes dynamic and source tracing.  
[Opus example: #16339](#)   [Sonnet example: #15693](#)   [Sonnet example: #16333](#)

# The reference loop and the Rust extension

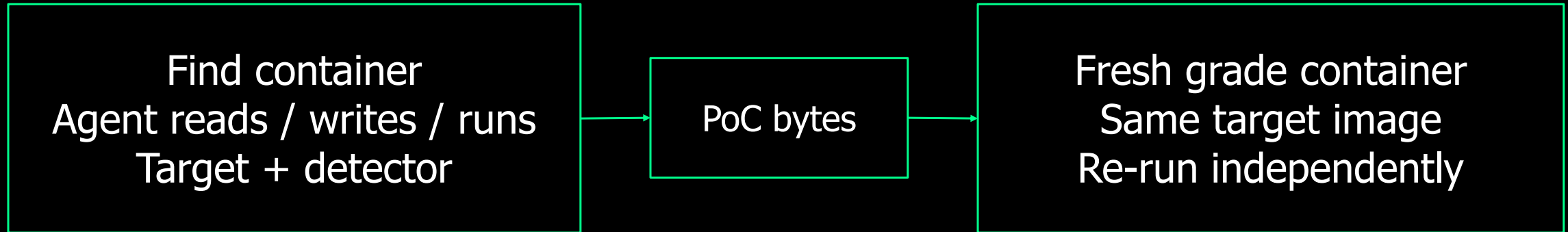
|                 | Reference harness                                  | rust-in-peace                                                              |
|-----------------|----------------------------------------------------|----------------------------------------------------------------------------|
| <b>Search</b>   | Threat-modelled review<br>+ parallel crash finders | Blind + threat model + history<br>Optional SAST worklists                  |
| <b>Evidence</b> | C/C++ memory failures<br>ASan + fresh grader       | Rust detectors + API / protocol tests<br>Optional find-to-fuzz + scorecard |
| <b>Feedback</b> | Verified findings<br>feed another patch / scan     | Refutations update the threat model,<br>rules, harnesses and variant seeds |

**Rust-in-peace extends the method while retaining the independent grader.**

The original already had threat modelling, parallel finders and patch verification.

[Reference methodology](#) [Rust methods](#) [Original defender loop](#)

# The execution path keeps the grader independent



Finder state stays behind. Validated evidence feeds report and patch stages.

**Only PoC bytes cross into the fresh grade container.**

Execution pipeline detail. The independent static research passes are a separate path.

[Agent sandbox](#) [Pipeline](#)

# The default Rust rules missed the questions

## 11 Rust rules

## 18 security queries

OpenGrep packs in this experiment

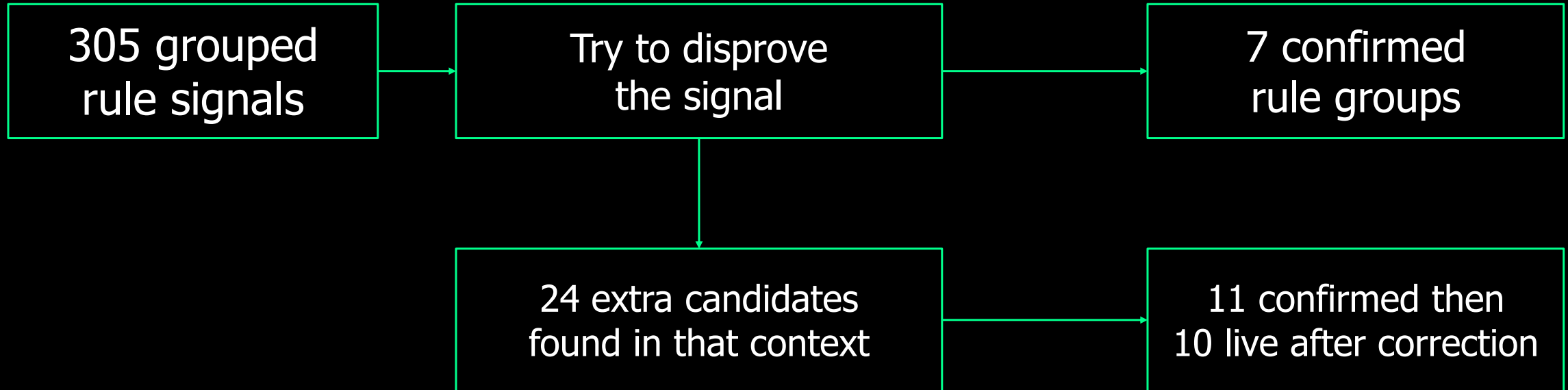
CodeQL rust-queries  
Zero hits on 6 of 7 parser crates

**The threat model and earlier findings supplied the missing questions.**

OpenGrep 1.26.0. CodeQL 2.26.1 / rust-queries 0.1.38. Counts are snapshot-specific.

[Tested default rule packs](#)

# The useful bugs appeared during refutation



**The scanner pointed to fewer confirmed hits than the investigation found along the way.**

Historical early experiment with five ast-grep enumerators. CodeQL joined later.  
Different populations: these counts cannot be compared as recall rates.

[Rules from research findings](#)

# SAST attribution: different units of observation

The second population was generated while investigating the first.

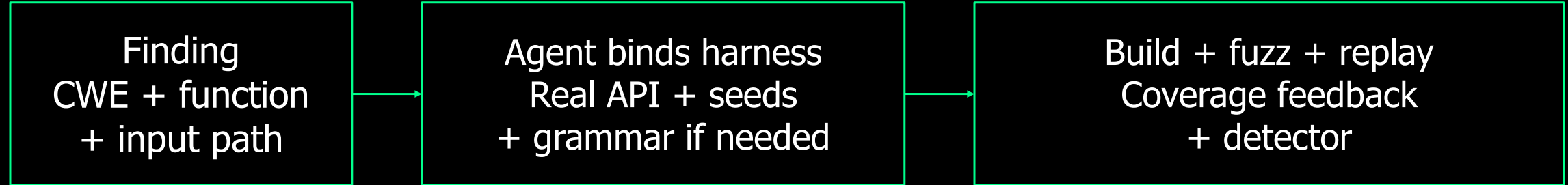
| Attribution             | Input population    | Historical result     |
|-------------------------|---------------------|-----------------------|
| Rule signal             | 305 grouped signals | 7 confirmed groups    |
| Alongside investigation | 24 extra candidates | 11 confirmed findings |

**Later correction: one alongside finding was works as intended, leaving 10 live.**

These are separate counts. No comparative precision or recall claim.

[E13: cells and candidates](#) [E13: 7 rule / 11 alongside](#)

# Guided fuzzing: a finding becomes a harness



## Choose the failure oracle

ASan / panic for byte parsers

Miri, TSan or compile-proof for other classes

**The agent guides the experiment. Coverage feedback guides libFuzzer.**

DVRA-003: 2/2 bridge runs reproduced one planted defect. Not two new vulnerabilities.

[Find → fuzz method](#) [DVRA-003 run](#)

# Guided fuzzing: the public DVRA-003 record

panic at parser/src/lib.rs:37:  
range start index 11 out of range  
for slice of length 10

Find: 2/3 runs found the planted crash.  
Bridge: 2/2 reproduced via byte\_parser.rs / ASan.  
Scorecard: exit 0.

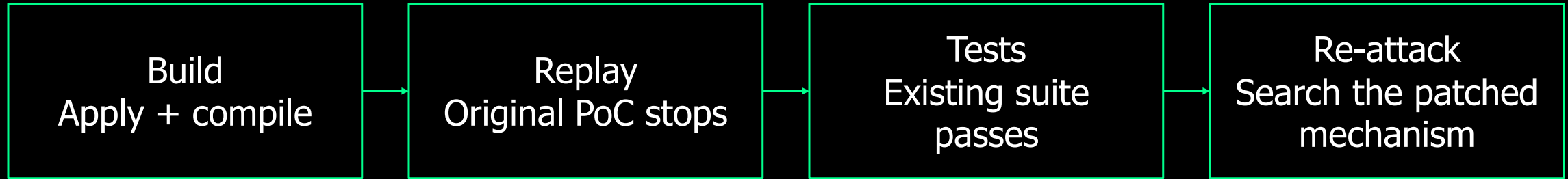


**The two successful bridge runs reproduce the same planted defect.**

Excerpt and results from the public target README. This is a historical record.

[DVRA-003 README](#)

# A patch must survive another experiment



Human review of the diff remains necessary.

**Stopping one reproducer does not establish that the whole mechanism is fixed.**

[Patch verification ladder](#)

# It worked on real crates

## object: the size cap was applied too late

Zstd output could exceed the declared  
uncompressed size.

The maintainer fixed the mechanism in  
#961.

### Public cases with fixes

|                     |                                    |
|---------------------|------------------------------------|
| object              | <a href="#">#961</a>               |
| lopdf               | <a href="#">#531 / #533</a>        |
| h2                  | <a href="#">#936</a>               |
| image               | <a href="#">#3095</a>              |
| ttf-parser          | <a href="#">#222–225, #234–236</a> |
| skrifa / fontations | <a href="#">#2012 / #2014</a>      |
| harfrust            | <a href="#">#411</a>               |

## Then the agents agreed on a very convincing bug.

Selected public cases with fixes. This is not a complete count of tested crates or confirmed findings.

[Public disclosure log](#) [object fix #961](#)

# A very convincing bug

## **modulus[0]**

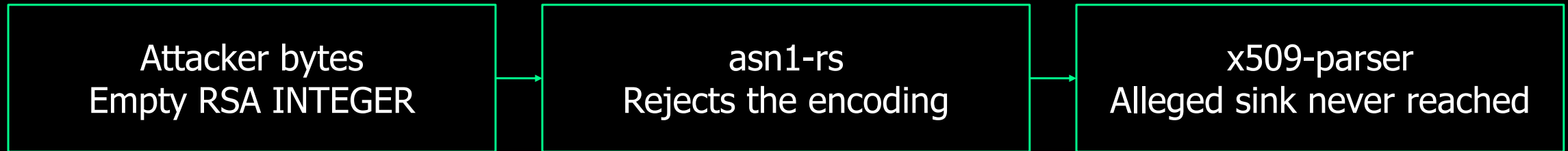
Candidate: an empty RSA INTEGER reaches this indexing operation.

Finder agrees. Triage agrees. Nearby guards make the missing check look suspicious.

**Does parsed attacker input actually reach it?**

[LESSONS L39: x509](#)

# Verify: the parser rejected the premise



**STOP**

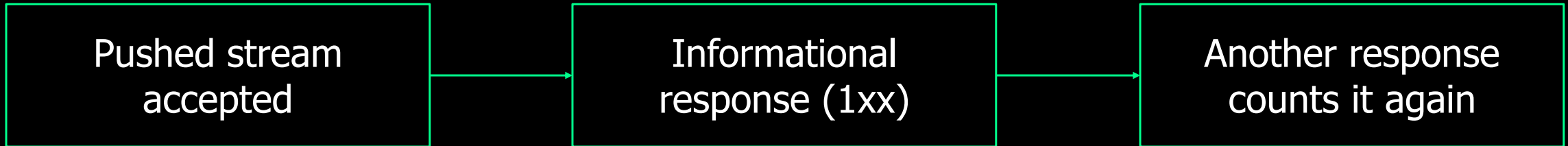
Manually constructing an empty value bypassed the parser guard.

**Agreement did not establish reachability. Reading the dependency changed the verdict.**

Opus retained the claim. Sonnet rejected it after opening the dependency. Local experiment log.

[LESSONS L39: dependency\\_guard](#)

## h2: server-push accounting becomes a client DoS



**One stream. Two accounting updates.**

Inconsistent state triggers an assertion panic.

**A hostile peer can trigger a panic when the client permits server push.**

Task failure vs process termination depends on the build and panic handling. Fixed by h2 #936.

[h2 fix #936](#)

# The consumer decides whether the path is open

The failure needs server push. A dependency version alone cannot answer reachability.

| Tested client path         | Server push            |
|----------------------------|------------------------|
| Direct h2 client           | Enabled by default     |
| hyper integration          | Disabled               |
| Deno WebSocket H2 fallback | Enabled before its fix |

**h2 #936 fixes accounting. Deno #36327 disables its unused push path.**

Pinned historical paths. No claim that all hyper, reqwest or tonic users expose this flaw.

[h2 #936](#) [Deno #36327](#)

# RustDesk reused the encryption keystream

## Encrypted remote desktop traffic

Client to peer  
Key K, nonce 0

**Same key**  
**Same nonce**

Peer to client  
Key K, nonce 0

$$C_1 \oplus C_2 = P_1 \oplus P_2$$

A relay or on-path observer can relate the plaintexts without knowing the key.

**The fix derives a different key for each direction.**

Known plaintext enables corresponding plaintext recovery. This is a confidentiality failure, not a claim of arbitrary decryption.

[Key-exchange fix #614](#)

[Integration #16326](#)

# RustDesk clipboard data became filesystem writes



## macOS: metadata and destination symlinks

A crafted clipboard transfer could redirect a write outside the intended destination.

## A second boundary

Incoming clipboard files bypassed the file-transfer permission toggle.

## Memory-safe parsing still needs path and permission checks.

The macOS case needs a peer-controlled clipboard transfer and a paste. Its fix predates our report by one day.

[macOS fix #15693](#)

[Permission fix #16333](#)

# RustDesk: nine confirmed, six fixes in master

9

findings acknowledged  
by the vendor

6

fixes merged  
into master

0

of those six fixes  
in stable 1.4.9

---

5 Aug  
Report sent

10 Aug  
All nine confirmed

23–26 Sep  
More fixes land

29 Sep  
Release still  
pending

**A maintainer response moved the work forward. Users still need a release.**

Six includes the independently fixed macOS case. Three findings remain private. Status checked 29 Sep 2026.

[Latest fix #16329](#)

[Stable 1.4.9](#)

[Disclosure record](#)

# Codex: the harness also has an attack surface

Untrusted repositories and MCP servers reach the harness.

| Boundary                | Reported impact                    | Condition                                  |
|-------------------------|------------------------------------|--------------------------------------------|
| Workspace write policy  | Hook execution outside the sandbox | A later Git operation outside the sandbox  |
| MCP OAuth metadata      | Launch of an OS URL handler        | User starts OAuth login to the MCP server  |
| MCP HTTP / SSE response | Client memory exhaustion           | A malicious server sends an unbounded body |

The tools available to the agent become part of the security boundary.

Six public reports. These cases have different prerequisites and do not all amount to a sandbox escape.

[Nested Git #37081](#)

[OAuth #37077](#)

[Response size #37080](#)

# The sandbox permitted a nested .git write

## CONTROL

.git/hooks/rip-canary  
Operation not permitted  
exit=1

## NESTED REPOSITORY

vendor/sub/.git/hooks/  
rip-canary  
exit=0

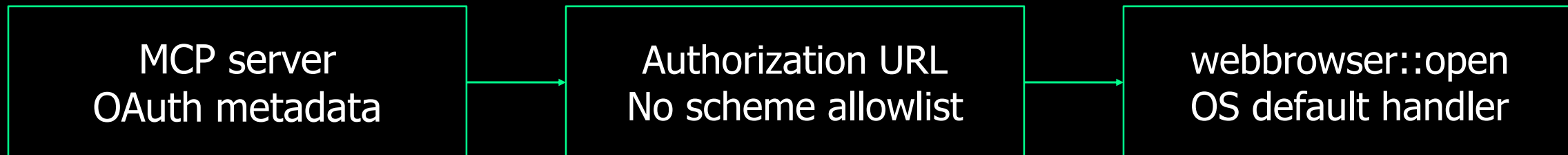
Exact path / result excerpts from the saved test

**The archived macOS / Seatbelt test proves the write.**

A stored canary file is evidence of a write. It is not a recording of later execution.

[Codex #37081](#)

# MCP OAuth trusted the server's URL scheme



**file: smb: search-ms: custom app schemes**

The saved test preserved non-web schemes in the generated URL.  
An OS-handler launch or SMB connection depends on the platform.

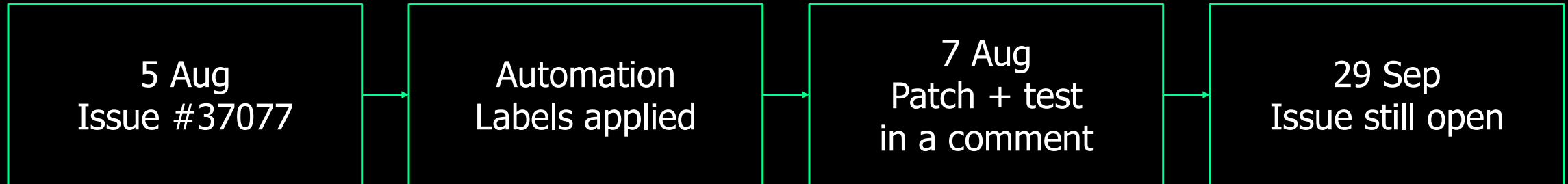
**A trusted tool call can carry an untrusted destination.**

Requires OAuth login to the MCP server. The test stopped before launching a handler. No demonstrated RCE claim.

[Issue #37077](#)

[Candidate fix + test](#)

# Waiting for a human

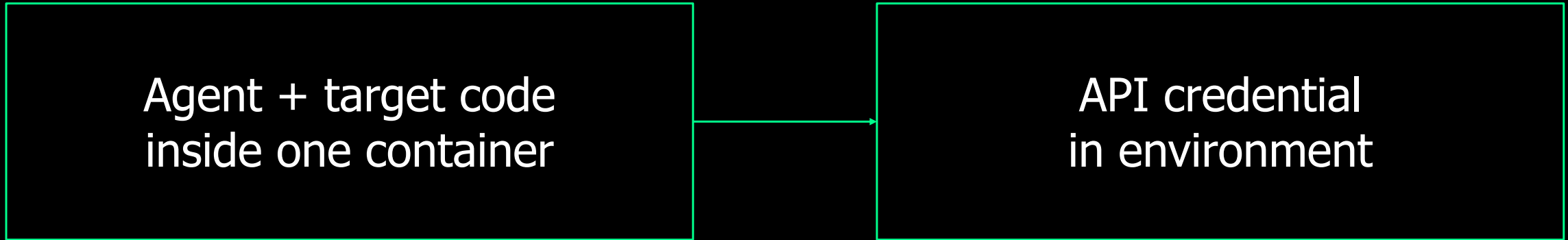


**I filed the issue. Robots labelled it. The patch is waiting for a public human response.**

A joke about the public intake trail. Automated labels do not establish technical review.

[Issue #37077](#) [Bot event](#) [Patch + test](#)

# Our own agent could read the credential



Architectural fix: inject credentials at a proxy outside the target environment.

**Restricting egress does not hide a credential from code in the same environment.**

Historical self-review. Credential-injecting proxy remained an architectural follow-up.

[Self-review F1](#) [Fix status](#)

# I sent those noisy reports

## Too much verification work landed on the maintainers.

Noisy batch. Wrong patch base. An LLM-written reply.  
The response included a temporary-ban message.



**“Your report should be concise and human written.”**

The policy PR predated my batch and merged during the episode.

[image #3086: response](#) [Policy #3068](#)

# My Chromium report reached an upstream fix



**BMP declares a large ICC profile.**

Allocation happened before validating that the profile range exists in the file.

**My report → an Edge engineer → image-rs fix → Chromium roll**

BMP is a specific causal chain, separate from the earlier noisy image-rs batch.

[Chromium report](#) [image-rs #3094 / #3095](#) [Chromium roll](#)

# object: an honest support boundary

## Maintainer's position (paraphrased)

Broad resistance to hostile inputs and DoS is outside the promised scope. Systematic hardening needs contributors.

## The specific fix landed

The maintainer fixed the Zstd mechanism in #961.

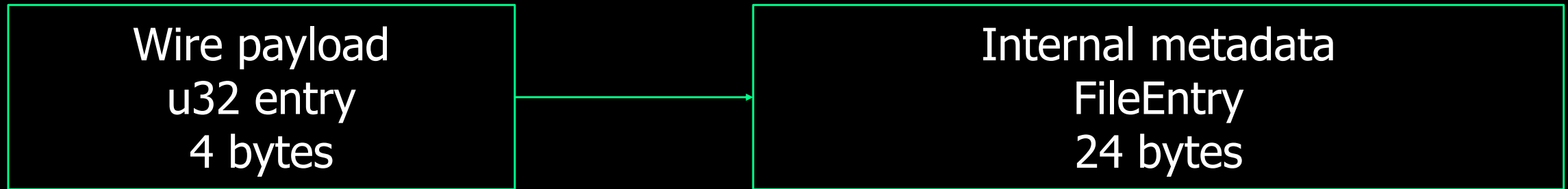
One fix cannot promise that every parser path tolerates hostile input.

**A security tool that adopts the crate still owns its own failure policy.**

[Maintainer response](#) [Fix #961](#)

# Rust in the kernel: the input was already bounded

## Rust Binder



The count passes the input bound. The expanded vector still requests physical contiguity.

**A bound on wire bytes does not bound the allocator request in the same way.**

Public patch example: Rust Binder metadata on a 64-bit system.

[Binder patch v2](#)

# Four changed lines, after checking the allocator

**900,000 entries**

FileEntry: ~20.6 MiB

Reservation: ~13.7 MiB

next-20260928

**KVec**

physically contiguous allocation

**KVVec**

vmalloc fallback for large allocations

**KVVec lets large allocations fall back to vmalloc.**

Two vectors changed. The patch leaves the close\_on\_free allocation separate.

[Patch and test record](#)

# The next dependency is a human

We have a reproducer, a patch and a review.

How does the fix reach the person running the software?

## Report

Give someone checkable evidence

## Release

Turn a merged patch into a package

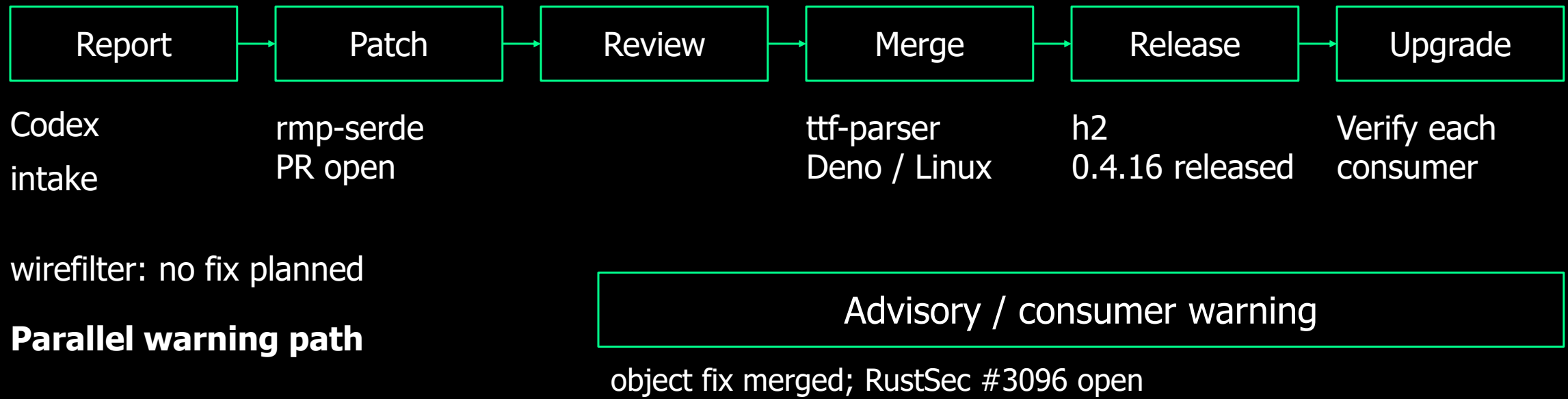
## Upgrade

Deliver it in the consuming product

**A report still has to become a shipped fix.**

[rust-in-peace](#)

# Ship: where a fix stops moving

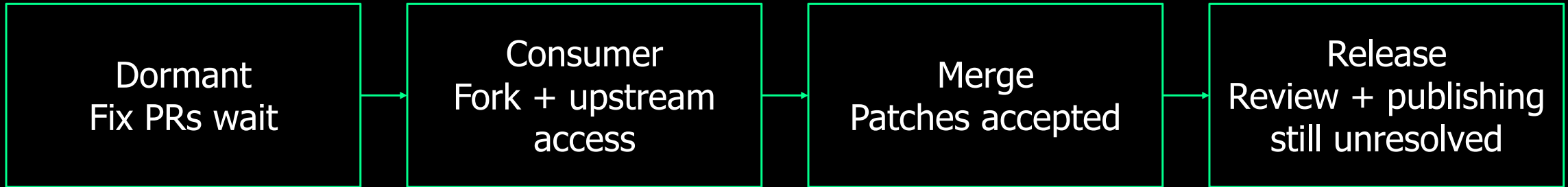


**An advisory is a parallel warning path. A release still needs consumer adoption.**

Status: 29 Sep 2026. Linux: integration trees. A merge does not establish deployed protection.

[rmp-serde #382](#) [ttf-parser #230](#) [RustSec #3096](#)

# The repository came back. The release did not.



Then: breaking changes, revert / archive discussion, narrower scope.

**GitHub access does not confer crates.io publishing rights or resolve review.**

RustSec had already marked ttf-parser unmaintained: RUSTSEC-2026-0192.

[ttf-parser #230](#) [Merged #222](#) [RUSTSEC-2026-0192](#)

# wirefilter: no fix planned for the retiring component

## Confirmed description. No fix planned.

Cloudflare ProdSec's response, as recorded in my public disclosure log: the component using this FFI layer was being retired.

**External consumers still need to assess their own use of the code.**

Author's account of a private response. This does not say all wirefilter development ended.

[Vendor response in disclosure log](#) [wirefilter #194](#)

# A merged fix can still be absent from RustSec

object #961  
FIX MERGED

RustSec #3096  
ADVISORY OPEN

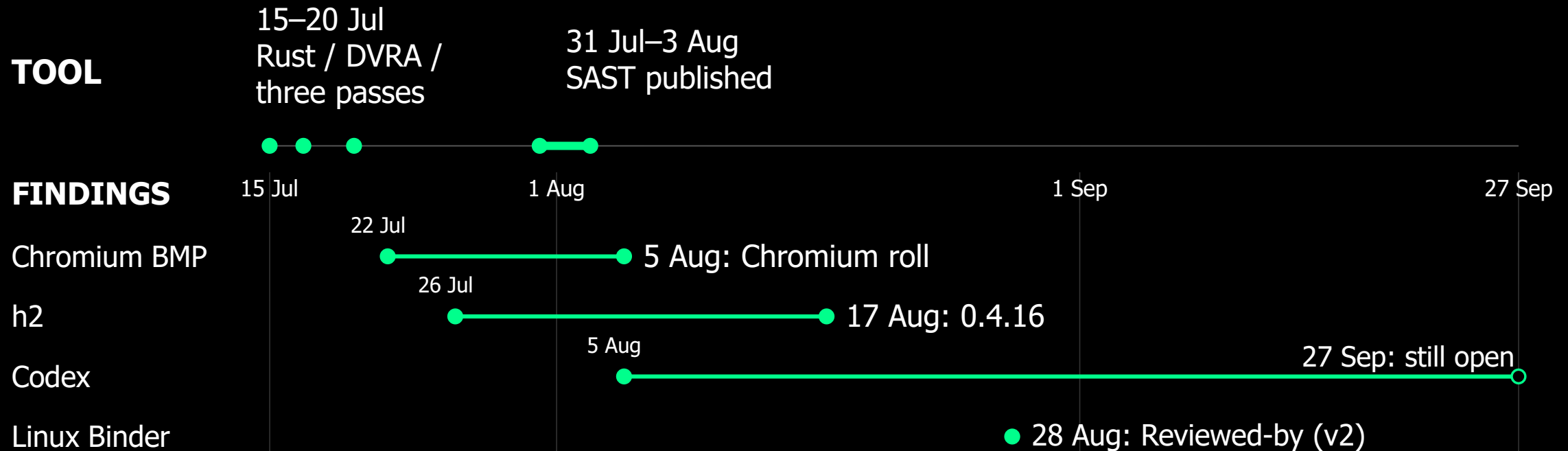
Upstream confirmation helps reject false reports.  
Disputed scope and review can also delay warnings.

**No advisory means no warning for this issue. It does not establish that the crate is healthy.**

object #961 merged. RustSec advisory PR #3096 remained open on 27 Sep 2026.

[object #961](#) [RustSec #3096](#) [Advisory policy](#)

# The project and its findings



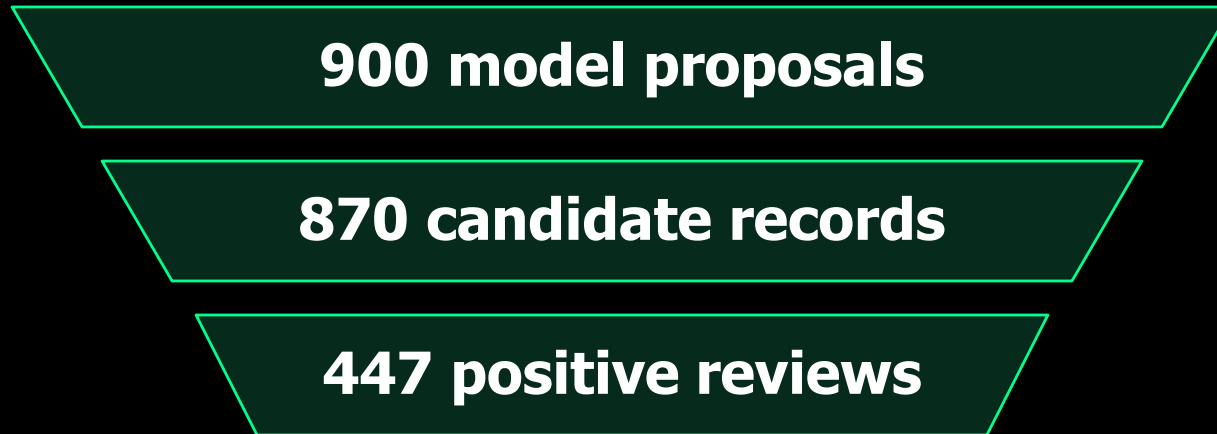
**Work on the tool and work on fixes continued in parallel.**

Selected milestones, Jul–Sep 2026. Tool dates are commits. Case endpoints have different meanings. Status: 27 Sep.

[Rust profile: 15 Jul](#) [SAST publication](#) [Public disclosure log](#)

# The finding funnel and the disclosure record

## 62 recovered runs



62 disputed; 308 rejected

53 gated, untested or incomplete

**69 findings have recorded dynamic PoCs.**

## All reported campaigns

**103**

reported finding groups

**46**

with an upstream fix

The two populations differ. No end-to-end conversion rate. Cross-run repeats remain on the left. Snapshot: 29 Sep 2026.

[Disclosure record](#)

[Latest RustDesk merge](#)

# 103 reported findings. 46 upstream fixes

| Outcome on 29 Sep                 | Finding groups | A fix can be at different stages              |
|-----------------------------------|----------------|-----------------------------------------------|
| Fixed / merged upstream           | 46             | h2                                            |
| Public, awaiting action           | 21             | 0.4.16 released                               |
| Private / kernel, awaiting action | 25             | RustDesk<br>6 in master, 0 in stable 1.4.9    |
| Closed, disputed or declined      | 11             | Linux Binder<br>char-misc-next and linux-next |

46 fixes upstream does not mean 46 fixes running on users' systems.

Counts use disclosure groups, including refunds. Upstream fixes: 45 user-space groups + 1 kernel integration-tree patch.

[Public disclosures](#)

[RustDesk #16329](#)

[Linux commit](#)

# “Just write the right prompt”

**~137 h**

elapsed agent activity

Overlapping intervals counted once

**1,173**

human prompts

615 subagents included in activity

**~210 h**

agent-hours

Parallel work adds across agents

**12.16B**

tokens

11.66B are cache reads

Preserved logs: both Claude profiles + Codex, 1 Jun–24 Sep 2026.

Hours include tools and waits. Neither human screen time nor pure inference time.

# Evidence should be cheap to check

|                    |                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------|
| <b>Researchers</b> | Minimal PoC, real entry path and build conditions. Help with a mechanism-level fix.     |
| <b>Maintainers</b> | Keep the evidence bar high. Let agents help triage without discarding real signal.      |
| <b>Consumers</b>   | Verify your configuration. Contribute tests, reviews and releases. Upgrade the product. |

**The useful outcome is a verified fix in the product people run.**

[oss-sec: research summary](#)

# Attackers don't wait for a merge

Attackers have the same tools as researchers.

They do not need our report policies, coordinated disclosure or PR approval to use a bug.

**Let's help each other stay ahead.**

Thank you to the maintainers. Selected details under embargo. Write-up coming. I swear.

[rust-in-peace](#) [oss-sec](#)